Hands-On 5

RAW SOCKET - ICMP

Sandi Russo
Corso di Laurea: Scienze Informatiche

7 aprile 2026

Indice

1 Introduzione **3**
1.1 Raw Socket 3
1.1.1 Differenza tra User Space e Kernel Space 3
1.2 Protocollo ICMP 3
1.3 Formati dei Messaggi ICMP 4

2 Definizione del problema **5**
2.1 Obiettivi Specifici 5
2.2 Riferimenti 6

3 Metodologia **6**
3.1 Creazione della Raw Socket 6
3.2 Costruzione del Pacchetto ICMP 6
3.3 Invio del Pacchetto ICMP 7
3.4 Ricezione della Risposta ICMP 7
3.5 Analisi della Risposta ICMP 7
3.6 Calcolo del Checksum 7

4 Presentazione dei risultati **7**
4.1 Inclusione delle Librerie 7
4.2 Definizione delle Costanti 8
4.3 Calcolo del Checksum 8
4.4 Creazione del Pacchetto ICMP 8
4.5 Gestione degli Argomenti 9
4.6 Creazione della Socket 9
4.7 Gestione del Timeout 9
4.8 Impostazione della Destinazione 10
4.9 Invio del Pacchetto ICMP 10
4.10 Ricezione della Risposta ICMP 10
4.11 Analisi della Risposta ICMP 11

5 Conclusione **11**

1 Introduzione

Il presente documento intende esplorare il funzionamento e l'utilizzo delle **raw socket** in combinazione con il protocollo **ICMP** (Internet Control Message Protocol). Lo studio approfondirà le caratteristiche tecniche di ICMP, le modalità di implementazione tramite raw socket in linguaggio C e l'analisi dei risultati ottenuti attraverso diversi grafici. La principale fonte di riferimento per il protocollo ICMP sarà l'*RFC 792*, mentre per le raw socket si farà riferimento alla documentazione di sistema *man*.

1.1 Raw Socket

I raw socket consentono di essere implementate ed operare nello user space e ad un livello più basso della pila di rete, permettendo la creazione e la manipolazione diretta dei pacchetti, inclusi gli header di protocollo (IP, TCP, UDP, ICMP, ecc...).
Vengono utilizzati per sviluppare strumenti di rete avanzati come sniffer, scanner, tool di sicurezza oppure protocolli personalizzati e per fare il *Packet Crafting*

1.1.1 Differenza tra User Space e Kernel Space

In un sistema operativo, esistono due spazi di memoria distinti in cui il codice può essere eseguito: lo user space e il kernel space. Questi spazi di memoria sono utilizzati per separare i processi di sistema dalle applicazioni degli utenti, garantendo così maggiore sicurezza e stabilità.

- **User Space**: questo è lo spazio di memoria in cui girano le applicazioni degli utenti. Le applicazioni in user space non hanno accesso diretto alle risorse hardware. Quando un'applicazione ha bisogno di accedere a risorse come il disco rigido, la rete o la memoria, deve effettuare una chiamata di sistema (system call) al kernel. Questo meccanismo di separazione garantisce che un'applicazione malfunzionante non possa compromettere la stabilità dell'intero sistema operativo.

- **Kernel Space**: questo è lo spazio di memoria riservato al kernel del sistema operativo, che ha accesso diretto a tutte le risorse hardware del sistema. Il kernel gestisce le risorse del sistema, come il processore, la memoria e i dispositivi di input/output. Le operazioni che richiedono un elevato livello di privilegio, come la gestione della memoria o la manipolazione dei pacchetti di rete a bassissimo livello, vengono eseguite nel kernel space.

1.2 Protocollo ICMP

Il protocollo ICMP è un protocollo di rete utilizzato per inviare messaggi di errore e informazioni operative indicate dalle macchine di rete, come router e host, per segnalare problemi relativi alla consegna dei pacchetti IP. Definito nello standard *RFC 792*, ICMP sfrutta il supporto di base dell'IP come se fosse un protocollo di livello superiore, ma è in realtà una parte integrante di IP e deve essere implementato da ogni modulo IP.

Nella pila ISO/OSI, il protocollo ICMP opera al livello 3 (Livello di Rete). Le raw socket, a loro volta, possono essere utilizzate per manipolare direttamente i pacchetti a questo livello, permettendo l'accesso e la modifica degli header di protocollo.

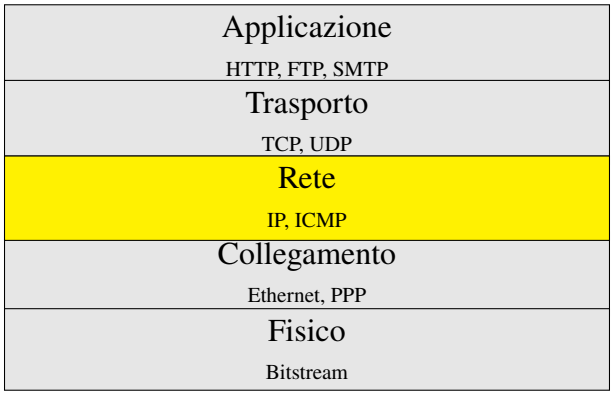

Figura 1: Pila ISO/OSI e Livello di Operazione delle Raw Socket

ICMP è utilizzato in diverse situazioni, ad esempio, quando un datagramma non può raggiungere la sua destinazione, quando il gateway non ha la capacità di buffer per inoltrare un datagramma, e quando il gateway può indirizzare l'host a inviare traffico su una rotta più breve. La finalità di questi messaggi di controllo è fornire feedback su problemi nell'ambiente di comunicazione, senza garantire la consegna assoluta dei datagrammi. I messaggi ICMP tipicamente segnalano errori nel processamento dei datagrammi, ma non vengono inviati messaggi ICMP su altri messaggi ICMP per evitare regressi infiniti di messaggi.

1.3 Formati dei Messaggi ICMP

I messaggi ICMP sono inviati utilizzando l'intestazione IP di base. Il primo ottetto della porzione dati del datagramma è un campo di tipo ICMP, il cui valore determina il formato dei dati rimanenti. I campi etichettati come "non utilizzati" sono riservati per estensioni future e devono essere zero quando inviati, ma i ricevitori non dovrebbero utilizzare questi campi eccetto che per includerli nel checksum.

- **Versione:** 4
- **IHL:** Lunghezza dell'intestazione internet in parole di 32 bit.
- **Tipo di Servizio:** 0
- **Lunghezza Totale:** Lunghezza dell'intestazione internet e dei dati in ottetti.
- **Identificazione, Flags, Offset di Frammento:** Utilizzati nella frammentazione.
- **Time to Live (TTL):** Tempo di vita in secondi, decrementato ad ogni passaggio attraverso una macchina.

- **Protocollo:** ICMP = 1
- **Checksum dell'Intestazione:** Complemento a uno della somma di complemento a uno di tutte le parole di 16 bit nell'intestazione.
- **Indirizzo Sorgente:** L'indirizzo del gateway o host che compone il messaggio ICMP.
- **Indirizzo Destinazione:** L'indirizzo del gateway o host a cui il messaggio deve essere inviato.

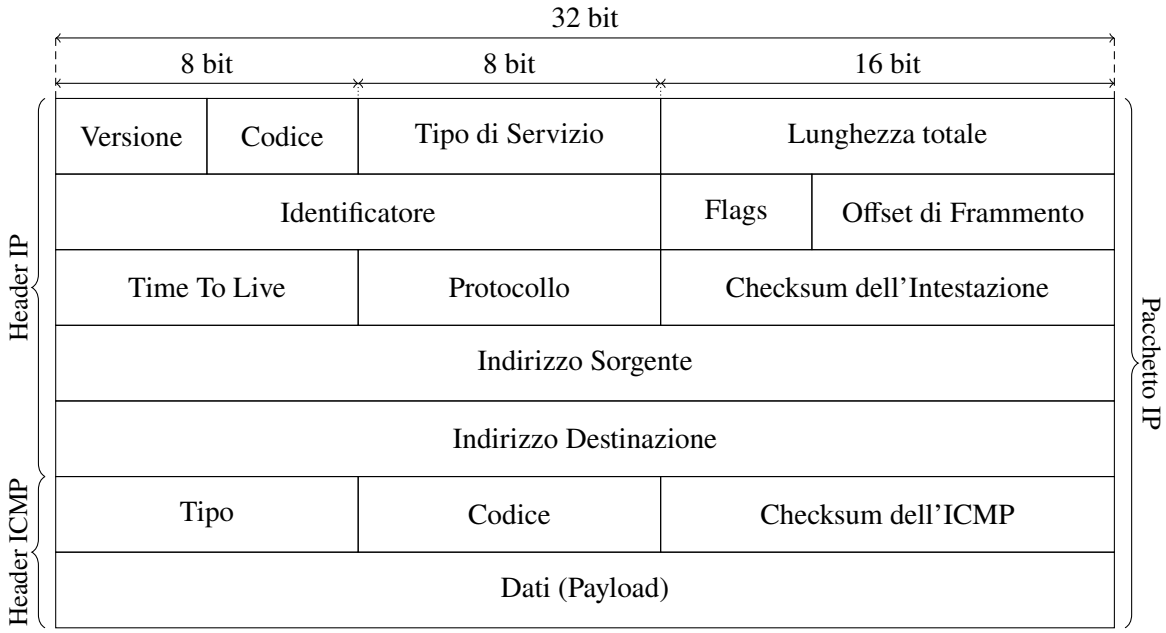

Figura 2: Formato del messaggio ICMP

2 Definizione del problema

L'obiettivo di questa relazione è comprendere come le raw socket possono essere utilizzate per inviare e ricevere messaggi ICMP, con particolare attenzione ai messaggi di richiesta ed eco (ping). Questo studio include la configurazione di una raw socket in un ambiente di sviluppo C, l'invio di pacchetti ICMP e l'analisi delle risposte ricevute.

2.1 Obiettivi Specifici

Gli obiettivi specifici di questa relazione includono:

- Comprendere la struttura dei pacchetti ICMP e il loro utilizzo pratico.
- Configurare e utilizzare raw socket per inviare messaggi ICMP.
- Analizzare le risposte ICMP ricevute e diagnosticare problemi di rete.

2.2 Riferimenti

Le principali fonti di riferimento per questo studio sono:

- RFC 792 - Internet Control Message Protocol
- Documentazione di sistema (‘man‘) per le raw socket e il protocollo ICMP

3 Metodologia

Per affrontare il problema di inviare e ricevere messaggi ICMP tramite raw socket, abbiamo seguito un approccio strutturato.

3.1 Creazione della Raw Socket

Per creare una raw socket, utilizziamo la funzione `socket()` con i parametri `AF_INET` (per IPv4), `SOCK_RAW` (per raw socket) e `IPPROTO_ICMP` (per il protocollo ICMP). Questo ci permette di creare una socket che opera a livello IP e utilizza il protocollo ICMP.

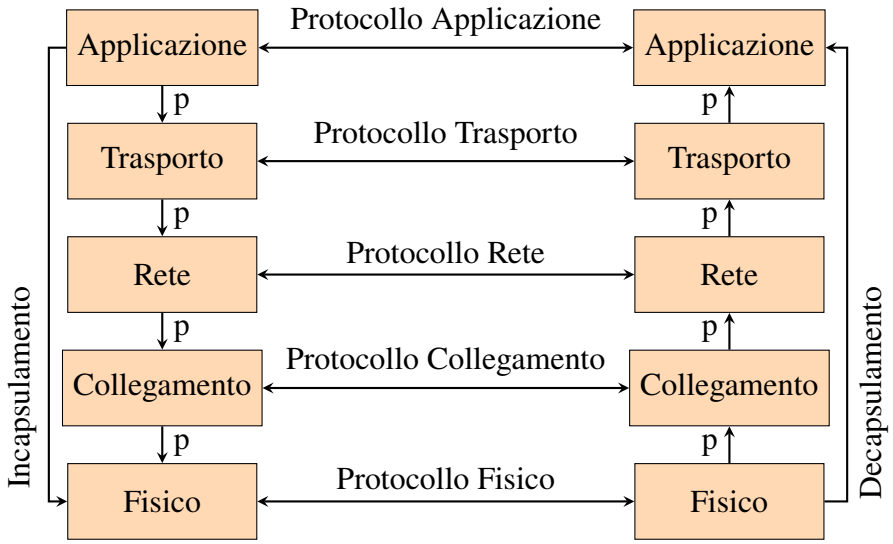

Figura 3: Schema del processo di incapsulamento e decapsulamento nelle reti

3.2 Costruzione del Pacchetto ICMP

Il pacchetto ICMP viene costruito manualmente. Questo include la configurazione dell’intestazione ICMP con i campi necessari come il tipo di messaggio, il codice, l’identificatore e il checksum. La funzione `build_icmp_packet()` si occupa di questa operazione.

3.3 Invio del Pacchetto ICMP

Una volta costruito il pacchetto ICMP, utilizziamo la funzione `sendto()` per inviare il pacchetto all'indirizzo di destinazione specificato. Questa funzione invia i dati della raw socket.

3.4 Ricezione della Risposta ICMP

Dopo aver inviato il pacchetto ICMP, attendiamo una risposta utilizzando la funzione `recvfrom()`. Questa funzione riceve i dati dalla raw socket e li memorizza in un buffer.

3.5 Analisi della Risposta ICMP

Una volta ricevuta la risposta, analizziamo il pacchetto per verificare che contenga una risposta valida (ICMP Echo Reply) e che l'identificatore corrisponda al PID del processo che ha inviato il ping. Utilizziamo la funzione `ntohs()` per convertire i valori dal formato di rete al formato host.

3.6 Calcolo del Checksum

Il calcolo del checksum è essenziale per garantire l'integrità dei pacchetti ICMP. Utilizziamo una funzione dedicata `compute_checksum()` per calcolare il checksum del pacchetto ICMP prima di inviarlo.

4 Presentazione dei risultati

4.1 Inclusione delle Librerie

Il primo passo consiste nell'includere le librerie necessarie per la gestione delle socket, del protocollo IP e del protocollo ICMP, nonché le librerie standard di input/output e gestione della memoria.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netinet/ip.h>
#include <netinet/ip_icmp.h> // La usiamo per gestire e utilizzare il
    protocollo icmp
#include <arpa/inet.h>
#include <sys/time.h> // La usiamo per gestire i timer
```

Listing 1: Inclusione delle librerie

4.2 Definizione delle Costanti

Definiamo alcune costanti che utilizzeremo nel programma, come la dimensione del pacchetto ICMP e il timeout per la risposta.

```
#define PACKET_SIZE 64
#define TIMEOUT_SEC 2
```

Listing 2: Definizione delle costanti

4.3 Calcolo del Checksum

La funzione `compute_checksum()` calcola il checksum per il pacchetto ICMP. Ma non l'andremo a studiare ed analizzare.

4.4 Creazione del Pacchetto ICMP

La funzione `build_icmp_packet()` crea un pacchetto ICMP di tipo ECHO REQUEST e lo riempie con i dati necessari, inclusi il tipo di messaggio, il codice, l'identificatore e il checksum.

```
void build_icmp_packet(struct icmphdr *icmp, int sequence) {
    icmp->type = ICMP_ECHO;  // ICMP_ECHO viene utilizzato dal comando
        ping per verificare se un host e' raggiungibile sulla rete
    icmp->code = 0;  // Codice impostato a 0 per l'echo reply
    icmp->un.echo.id = htons(getpid());  // ID univoco basato sull'ID del
        processo
    icmp->checksum = 0;

    // Aggiunge un payload qualunque
    char *data = (char *)(icmp + 1);  // Puntatore all'area successiva
        alla struttura ICMP
    memset(data, 'A', PACKET_SIZE - sizeof(*icmp));  // Riempie il payload
         con 'A'
    icmp->checksum = compute_checksum(icmp, PACKET_SIZE);  // Calcola il
        checksum
}
```

Listing 3: Creazione del pacchetto ICMP

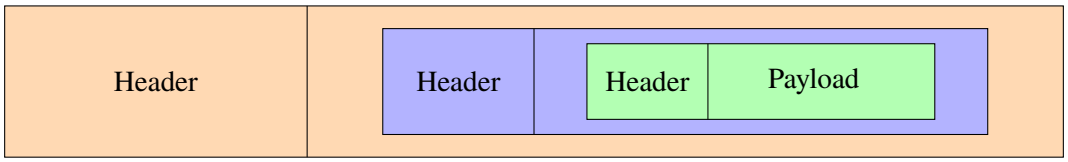

Figura 4: Schema di incapsulamento con header e payload

4.5 Gestione degli Argomenti

Il programma verifica che il numero di argomenti passati sia corretto.

```
int main(int argc, char *argv[]) {
    if (argc != 2) {  // Verifica che il numero di argomenti sia corretto
        printf("Utilizzo: %s <indirizzo IP>\n", argv[0]);
        exit(1);
    }
```

Listing 4: Gestione degli argomenti

4.6 Creazione della Socket

Creiamo una raw socket utilizzando il protocollo ICMP. Se la creazione della socket fallisce, il programma mostra un messaggio di errore e termina.

```
    int sock;
    struct sockaddr_in dest;  // Struttura per rappresentare gli indirizzi IPV4
    char packet[PACKET_SIZE];  // Inizializzazione del packet, specificandone la dimensione (header + payload)

    struct icmphdr *icmp = (struct icmphdr *)packet;  // Casting del packet alla struttura dell'header ICMP

    if ((sock = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP)) < 0) {  // Creazione della socket
        perror("socket");
        exit(1);
    }
```

Listing 5: Creazione della socket

4.7 Gestione del Timeout

Configuriamo un timeout per la ricezione della risposta ICMP. Questo è utile per evitare che il programma resti in attesa indefinitamente.

```
    struct timeval tv;  // Struttura usata per definire un timeout in secondi e microsecondi
    tv.tv_sec = TIMEOUT_SEC;  // Imposta il timeout con la define
    tv.tv_usec = 0;  // Non utilizzo microsecondi

    // Imposta il timeout sulla socket
    setsockopt(sock, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));
```

Listing 6: Gestione del timeout

4.8 Impostazione della Destinazione

Configuriamo l'indirizzo di destinazione (host a cui invieremo il ping) e azzeriamo la struttura dati di destinazione.

```
    memset(&dest, 0, sizeof(dest));  // Azzera la struttura dati di destinazione
    dest.sin_family = AF_INET;  // Utilizziamo IPv4
    dest.sin_addr.s_addr = inet_addr(argv[1]);  // Converte l'indirizzo IP da stringa a binario
```

Listing 7: Impostazione della destinazione

4.9 Invio del Pacchetto ICMP

Costruiamo il pacchetto ICMP utilizzando la funzione `build_icmp_packet()`.

```
    build_icmp_packet(icmp, 1);  // Costruisce il pacchetto ICMP
    printf("Invio ping a %s...\n", argv[1]);

    if (sendto(sock, packet, PACKET_SIZE, 0, (struct sockaddr *)&dest, sizeof(dest)) <= 0) {  // Invio del pacchetto ICMP
        perror("sendto");
        close(sock);
        exit(1); }
```

Listing 8: Invio del pacchetto ICMP

4.10 Ricezione della Risposta ICMP

Riceviamo la risposta ICMP dalla socket e la memorizziamo in un buffer.

```
    char recv_buffer[1024];  // Buffer per la risposta del ping
    struct sockaddr_in src;  // Struttura per rappresentare gli indirizzi IPV4
    socklen_t src_len = sizeof(src);  // Calcolo della dimensione della struttura

    int bytes = recvfrom(sock, recv_buffer, sizeof(recv_buffer), 0, (struct sockaddr *)&src, &src_len);
    if (bytes < 0) {  // Nessuna risposta
        perror("Nessuna risposta");
        close(sock);
        exit(1);
    }
```

Listing 9: Ricezione della risposta ICMP

4.11 Analisi della Risposta ICMP

Analizziamo il pacchetto ricevuto per verificare che contenga una risposta valida.

```
    struct iphdr *ip_hdr = (struct iphdr *)recv_buffer;  // Cast del buffer ricevuto alla struttura IP header
    struct icmphdr *icmp_reply = (struct icmphdr *)(recv_buffer + (ip_hdr->ihl * 4));  // Puntatore all'header ICMP

    if (icmp_reply->type == ICMP_ECHOREPLY && ntohs(icmp_reply->un.echo.id) == getpid()) {  // Verifica se il pacchetto e' una risposta valida
        printf("Ricevuta risposta da %s: seq=%d, TTL=%d\n",
               inet_ntoa(src.sin_addr),  // Converte l'indirizzo IP del mittente da binario a stringa
               ntohs(icmp_reply->un.echo.sequence),  // Converte il numero di sequenza nel formato dell'host
               ip_hdr->ttl);  // Mostra il TTL del pacchetto
    } else {
        printf("Risposta non valida\n");
    }

    close(sock);
    return 0;}
```

Listing 10: Analisi della risposta ICMP

5 Conclusione

In questa relazione, abbiamo descritto la creazione di una raw socket, la costruzione di un pacchetto ICMP, l'invio e la ricezione di questo pacchetto, e l'analisi della risposta ricevuta. Attraverso l'uso di grafici e schemi, abbiamo illustrato il processo di incapsulamento e decapsulamento nei livelli ISO/OSI e la struttura gerarchica dei pacchetti di rete. Questi strumenti visivi hanno aiutato a comprendere meglio il flusso dei dati e l'interazione tra i diversi livelli di protocollo.
I risultati ottenuti dimostrano l'efficacia delle raw socket nel manipolare direttamente i pacchetti di rete, offrendo un controllo granulare che è essenziale per sviluppare strumenti di rete avanzati come sniffer, scanner e tool di sicurezza. Tuttavia, l'uso delle raw socket richiede privilegi elevati e una comprensione approfondita dei protocolli di rete.
In conclusione, l'approccio presentato in questa relazione può essere esteso e adattato per altre applicazioni di rete, offrendo una base solida per ulteriori studi e sviluppi nel campo della gestione e dell'analisi dei pacchetti di rete.

```
sudo ./icmp 8.8.8.8
Invio ping a 8.8.8.8...
Ricevuta risposta da 8.8.8.8: seq=0, TTL=112
```

Listing 11: Esecuzione del programma di ping